(1) a)

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-g(x_i)|^2} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-y_i+y_i-g(x_i)|^2}$$

$$\leq \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(|f^*(x_i)-y_i| + |y_i-g(x_i)|\right)^2} =$$

$$= \sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-y_i|^2 + \frac{1}{n}\sum_{i=1}^{n}|y_i-g(x_i)|^2 + \frac{2}{n}\sum_{i=1}^{n}|f^*(x_i)-y_i||y_i-g(x_i)|}$$

$$\leq \sqrt{\left(\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-y_i|^2} + \sqrt{\frac{1}{n}\sum_{i=1}^{n}|y_i-g(x_i)|^2}\right)^2} = \underbrace{\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-y_i|^2}}_{\leq \varepsilon} + \sqrt{\frac{1}{n}\sum_{i=1}^{n}\underbrace{|y_i-g(x_i)|}_{e_i}{}^2}$$

by Cauchy Schwartz:

$$\sum_{i=1}^{n}|u_i||v_i| \leq \sqrt{\sum_{i=1}^{n}|u_i|^2}\sqrt{\sum_{i=1}^{n}|v_i|^2}$$

$$\leq \varepsilon + \sqrt{\frac{1}{n}\sum_{i=1}^{n}\underbrace{|e_i|^2}_{\leq E}} \leq \varepsilon + \sqrt{\frac{1}{n}\sum_{i=1}^{n}E^2} = \varepsilon + E, \text{ and letting}$$

$h(E) = E$, we get:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-g(x_i)|^2} \leq \varepsilon + h(E) = \varepsilon + E, \text{ which}$$

demonstrates such $h(E)$ independent of ε exists $\square$.

b) Let $x, y \in \mathbb{R}^d$, then:

$$|\phi((W_j^*)^T x + b_j^*) - \underbrace{\phi}_{\max\{0, \text{input}\}}((W_j^*)^T y + b_j^*)| =$$

$$= |\max\{0, (W_j^*)^T x + b_j^*\} - \max\{0, (W_j^*)^T x + b_j^*\}|.$$

Next consider: $\max\{0,a\} - \max\{0,b\}$, 4 cases:

$a<0, b<0$: $|\max\{0,a\} - \max\{0,b\}| = 0 \leq |a-b|$ ← always ≥ 0

$a>0, b<0$: $|\max\{0,a\} - \max\{0,b\}| = |a| \leq |a-b|$

$a<0, b>0$: $|\max\{0,a\} - \max\{0,b\}| = |-b| \leq |b-a| = |a-b|$ (since $-b \geq 0$)

$a>0, b>0$: $|\max\{0,a\} - \max\{0,b\}| = |a-b| \leq |a-b|$ ($-a \geq 0$). So it holds:

$|\max\{0,a\} - \max\{0,b\}| \leq |a-b|$.

Therefore:

$$|\phi((W_j^*)^T x + b_j^*) - \phi((W_j^*)^T y + b_j)| \leq |(W_j^*)^T x + b_j - ((W_j^*)^T y + b_j)|$$

$$= |(W_j^*)^T (x-y)| = \left|\sum_{k=1}^{d} W_{j,k}^* (x_k - y_k)\right| \leq \sum_{k=1}^{d} |W_{j,k}^*||x_k - y_k|$$

$$= \sum_{k=1}^{d} |W_{j,k}^*| \sqrt{(x_k - y_k)^2} \leq \sum_{k=1}^{d} |W_{j,k}^*| \underbrace{\sqrt{\sum_{i=1}^{d} (x_k - y_k)^2}}_{\text{L-2 norm}} = \underbrace{\left(\sum_{k=1}^{d} |W_{j,k}^*|\right)}_{:=L} \|x-y\|_2$$

$= L\|x-y\|_2$. So $\phi((W_j^*)^T x + b_j)$ is Lipschitz by definition, with Lipschitz constant $L = \left(\sum_{k=1}^{d} |W_{j,k}^*|\right)$.

(1) c) The original relu is: $\phi(z) = \max(0,z)$. Then, define:

$$\phi_\sigma(z) = \int_{-\infty}^{\infty} \phi(z-t)\, p_\sigma(t)\, dt \underset{\text{by definition}}{=} \mathbb{E}\left[\phi(z-Z)\right]$$

($p_\sigma(t)$: gaussian pdf; Z: gaussian RV.)

i.e. $p_\sigma(t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{t^2}{2\sigma^2}\right\}$

Next: $|f^*(x_i) - f^*_\sigma(x_i)| = |(V^*)^T \phi(W^* x_i + b^*) - (V^*)^T \phi_\sigma(W^* x_i + b^*)|$

$= \left|(V^*)^T \left(\phi(W^* x_i + b^*) - \mathbb{E}\left[\phi(W^* x_i + b^* - Z)\right]\right)\right| =$

$= \left|(V^*)^T \mathbb{E}\left[\phi(W^* x_i + b^*) - \phi(W^* x_i + b^* - Z)\right]\right| =$

$= \left|\sum_{j=1}^{m} (V_j^*)\, \mathbb{E}\left[\phi(W^* x_i + b^*) - \phi(W^* x_i + b^* - Z)\right]_j\right| \underset{\text{triangle ineq and } |\mathbb{E}[X]| \le \mathbb{E}|X|}{\le} \sum_{j=1}^{m} |V_j^*|\, \mathbb{E}\left|\phi(W^* x_i + b^*) - \phi(W^* x_i + b - Z)\right|_j$

$\underset{\text{from before}}{\le} \sum_{j=1}^{m} |V_j^*|\, \underbrace{\mathbb{E}|Z|_j}_{\text{gaussian RV}} = \sum_{j=1}^{m} |V_j^*|\, \underbrace{\sigma\sqrt{\frac{2}{\pi}}}_{\mathbb{E}|Z| \text{ when } Z\sim N(0,\sigma^2)} = \sigma\sqrt{\frac{2}{\pi}}\, \|V^*\|_1 \underset{\|V^*\|_1 \le \sqrt{m}\,\|V^*\|_2 \text{ by Cauchy-Schwartz}}{\le} \sigma\sqrt{m}\sqrt{\frac{2}{\pi}}\, \|V^*\|_2$

So: $|f^*(x_i) - f^*_\sigma(x_i)| \le \sigma\sqrt{\frac{2m}{\pi}}\, \|V^*\|_2$.

Then, since this holds for all x_i:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} |f^*(x_i) - f^*_\sigma(x_i)|^2} \le \sqrt{\frac{1}{n}\cdot \sum_{i=1}^{n} \frac{\sigma^2 \cdot 2m}{\pi}\, \|V^*\|_2^2} = \sigma\sqrt{\frac{2m}{\pi}}\, \|V^*\|_2,$$

establishing the desired bound (i.e. a valid upper bound).

d) First, by a) and c):

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f_\sigma^*(x_i)-g(x_i)|^2} \le \underbrace{\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f_\sigma^*(x_i)-f^*(x_i)|^2}}_{\le \sigma\sqrt{2m/\pi}\,\|v^*\|_2} + \underbrace{\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f^*(x_i)-g(x_i)|^2}}_{\le \varepsilon + h(E)}$$

$$\le \sigma\sqrt{\frac{2m}{\pi}}\,\|v^*\|_2 + \varepsilon + \underbrace{h(E)}_{=E}.$$

Now set: $\sigma\sqrt{\frac{2m}{\pi}}\,\|v^*\|_2 + \varepsilon + E \le 2\varepsilon \Leftrightarrow \sigma\sqrt{\frac{2m}{\pi}}\,\|v^*\|_2 + E \le \varepsilon.$

Let: $E \le \frac{\varepsilon}{2}$, $\sigma\sqrt{\frac{2m}{\pi}}\,\|v^*\|_2 \le \frac{\varepsilon}{2} \Rightarrow \sigma \le \frac{\varepsilon}{2\|v^*\|_2}\sqrt{\frac{\pi}{2m}}$.

So the following upperbounds: $E \le \frac{\varepsilon}{2}$, $\sigma \le \frac{\varepsilon}{2\|v^*\|_2}\sqrt{\frac{\pi}{2m}}$ are sufficient to ensure:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}|f_\sigma^*(x_i)-g(x_i)|^2} \le 2\varepsilon.$$

(2) a)

$$\mathbb{E}_{u_1,\dots,u_N}[\tilde{h}(x)] = \mathbb{E}_{u_1,\dots,u_N}\left[\frac{1}{N}\sum_{j=1}^{N}\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle)\right] =$$

$$= \frac{1}{N}\sum_{j=1}^{N}\mathbb{E}_{\substack{\vec{u}\\ \vdots \\ u_1,\dots,u_N}}\left[\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle)\right] \underset{\text{iid}}{=} \mathbb{E}_{\vec{u}}\left[\tilde{w}(u)\,\sigma(\langle u,x\rangle)\right] \text{ for } u\sim p_u$$

u_j's are iid, so each expectation is the same

By definition:

$$\mathbb{E}_{u_1,\dots,u_N}[\tilde{h}(x)] = \mathbb{E}_{\vec{u}}\left[\tilde{w}(u)\,\sigma(\langle u,x\rangle)\right] = \int_u p_u(u)\,\tilde{w}(u)\,\sigma(\langle u,x\rangle)\,du =$$

(we want this to hold)

$$h(x) = \int_u d(u)\,\sigma(\langle u,x\rangle)\,du \Leftrightarrow p_u(u)\,\tilde{w}(u) = d(u) \Leftrightarrow$$

$$\tilde{w}(u) = \frac{d(u)}{p_u(u)} \quad \text{where } p_u \text{ is the probability density.}$$

Now, let $\tilde{w}(u) = \frac{d(u)}{p_u(u)}$. Then:

$$\mathbb{E}_{u_1,\dots,u_N}\left[|\tilde{h}_u(x) - h(x)|\right] = \int_u p_u(u)\sqrt{(\tilde{h}_u(x)-h(x))^2}\,du = \int_u \underbrace{\sqrt{p_u(u)}}_{d(u)}\underbrace{\sqrt{p_u(u)(\tilde{h}_u(x)-h(x))^2}}_{f(u)}\,du$$

$$\underset{\text{Cauchy Schwarz}}{\leq} \sqrt{\int_u p_u(u)(\tilde{h}_u(x)-h(x))^2\,du}\;\underbrace{\sqrt{\int_u p_u(u)\,du}}_{=1} = \sqrt{\mathbb{E}_{u_1,\dots u_N}[(\tilde{h}_u(x)-h(x))^2]} =$$

$\int f(u)g(u)\,du \leq \sqrt{\int f(u)^2\,du}\sqrt{\int g(u)^2\,du}$

$$= \sqrt{\mathbb{E}_{u_1,\dots,u_N}\left[\left(\frac{1}{N}\sum_{j=1}^{N}\left(\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle)-h(x)\right)\right)^2\right]} = \sqrt{\frac{1}{N^2}\mathbb{E}_{\vec{u}}\left[\left(\sum_{j=1}^{N}\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle)-h(x)\right)^2\right]}$$

$$= \frac{1}{N}\sqrt{\sum_{j=1}^{N}\mathbb{E}_{\vec{u}}\left[\left(\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle)-h(x)\right)^2\right]}$$

the sum goes out because $\tilde{w}(u_j)\,\sigma(\langle u_j,x\rangle) \perp \tilde{w}(u_k)\,\sigma(\langle u_k,x\rangle)$ for $j\neq k$, so by unbiasedness $\mathbb{E}=0$.

to clarify $\mathbb{E}[(\sum W_j)^2] = \mathbb{E}[\sum_{j=1}^{N} W_j^2] + \mathbb{E}[\sum_{j\neq k} W_j V_k]$,

but for $j\neq k$: $\mathbb{E}[\sum_{j\neq k} W_j W_k] = \sum_{j\neq k}\underbrace{\mathbb{E}[W_j]}_{0}\underbrace{\mathbb{E}[W_k]}_{0} = 0$.

Next, note that: $\mathbb{E}_{\vec{w}}\left[\left(\tilde{w}(u_j)\,\sigma(\langle u_j, x\rangle) - h(x)\right)^2\right] =$

$= \mathbb{E}_{\vec{w}}\left[\left(\tilde{w}(u_j)\,\sigma(\langle u_j, x\rangle)\right)^2\right] - 2\underbrace{\mathbb{E}_{\vec{w}}\left[w(u_j)\,\sigma(\langle u_j, x\rangle)\right]}_{= \mathbb{E}_{\vec{w}}[\hat{h}(x)] = h(x)} h(x) + h(x)^2 =$

$= \mathbb{E}_{\vec{w}}\left[\left(\tilde{w}(u_j)\,\sigma(\langle u_j, x\rangle)\right)^2\right] - h(x)^2 \leq \mathbb{E}_{\vec{w}}\left[\left(\tilde{w}(u_j)^2\,\sigma(\langle u_j, x\rangle)^2\right] \leq$

$= -2h(x)^2 + h(x)^2$

$0 \leq \sigma \leq 1$

$|\tilde{w}(u)| \leq A$

$\leq \mathbb{E}_{\vec{w}}[A^2 \cdot 1] = A^2.$

By that fact, and previous inequalities:

$$\mathbb{E}_{u_1,\dots,u_N}\left[\left|\tilde{h}_u(x) - h(x)\right|\right] \leq \frac{1}{N}\sqrt{\sum_{j=1}^{N} A^2} = \frac{1}{N}\sqrt{NA^2} = \frac{A\sqrt{N}}{\sqrt{N}\sqrt{N}} = \frac{A}{\sqrt{N}},$$ so:

$$\mathbb{E}_{u_1,\dots,u_N}\left[\left|\tilde{h}_u(x) - h(x)\right|\right] \leq \frac{A}{\sqrt{N}} \quad \forall x \in \mathbb{R}^d$$ (since this holds for any x),

and:

$$\mathbb{E}_{u_1,\dots,u_N}\left[\tilde{h}(x)\right] = h(x) \quad \text{when } \tilde{w}(u) = \frac{\alpha(u)}{p_u(u)}. \quad \square$$

b) $L(\tilde{h}_u) = \mathbb{E}_{u, x\sim p_x}\left|\tilde{h}_u(x) - h(x)\right| = \mathbb{E}_{x\sim p_x}\left[\mathbb{E}_u\left[\left|\tilde{h}_u(x) - h(x)\right|\right]\right] \leq$

($=$: by definition; $=$: by law of iterated expectations; $\leq$: by a)

$\leq \mathbb{E}_{x\sim p_x}\left[\frac{A}{\sqrt{N}}\right] = \frac{A}{\sqrt{N}}$, so $L(\tilde{h}_u) \leq \frac{A}{\sqrt{N}}$ $\square$.

(2) c) The ANN's architecture is an MLP with 1 hidden layer, and sigmoid activation, and one output layer. In detail:

x: d-dimensional input

U: First linear layer of the MLP, transforms x into N-dimensional space.

σ: Activations applied to outputs of linear layer U, sigmoid applied element-wise.

w: Output layer, single neuron, combines activations.

So the architecture is:

$$x \xrightarrow[\text{input}]{} \underset{(\text{weights } U)}{\text{hidden layer}} \rightarrow \underset{(\text{sigmoid})}{\text{activation}} \rightarrow \underset{(\text{weights } w)}{\text{output layer}} \rightarrow f(x)$$

Total parameters $= \underbrace{N \cdot d}_{U} + \underbrace{N}_{w} = N(d+1)$.

d) $\tilde{h}_w$ is: $\tilde{h}_w(x) = \frac{1}{N}\sum_{j=1}^{N} \tilde{w}(w_j)\sigma(\langle w_j, x\rangle)$, $f(x) = w^T\sigma(Ux) = \sum_{j=1}^{N} w_j \sigma(w_j^T x)$

Since $w_j \sim p_w$ and frozen (as stated), if $w_j = \frac{\tilde{w}(w_j)}{N}$, $\tilde{h}_w(x) = f(x)$.

Then, from a): $|\tilde{w}(w)| \leq A \Rightarrow |w_j| = \left|\frac{\tilde{w}(w_j)}{N}\right| = \frac{|\tilde{w}(w_j)|}{N} \leq \frac{A}{N}$, and so:

$\|w\|_1 = \sum_{j=1}^{N} |w_j| = \sum_{j=1}^{N} \frac{|\tilde{w}(w_j)|}{N} \leq \sum_{j=1}^{N} \frac{A}{N} = N \cdot \frac{A}{N} = A$. So:

$p = 1$, $B = A$ ensure that $\tilde{h}_w$ is a feasible solution to the defined optimization problem.

c) On Piazza, the instructors confirmed asking about empirical loss,

so:

$$\hat{L}(h_w^*) = \frac{1}{n}\sum_{i=1}^{n} |y_i - h_w^*(x_i)| = \frac{1}{n}\sum_{i=1}^{n} |y_i - (w^*)^T \sigma(U x_i)| \leq$$

$$\leq \frac{1}{n}\sum_{i=1}^{n} |y_i - \tilde{h}_w(x_i)| = \underbrace{\hat{L}(\tilde{h}_w)}_{\text{empirical loss}}$$

since w^* is a minimizer, and $\tilde{h}_w$ is a feasible solution.

So:

$$L(h_w^*) = \mathbb{E}_{D,w}\left[\hat{L}(h_w^*)\right] \leq \mathbb{E}_{D,w}\left[\hat{L}(\tilde{h}_w)\right] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{D,w}\left[|\tilde{h}_w(x) - h(x)|\right]$$

by above

$$\leq \frac{1}{n}\sum_{i=1}^{n} \frac{A}{\sqrt{N}} = \frac{1}{n} \cdot \frac{nA}{\sqrt{N}} = \frac{A}{\sqrt{N}},$$

by result in b), and iid data

so: $L(h_w^*) \leq \frac{A}{\sqrt{N}}$ □.

Solution for Assignment 02 Question 3

Setup

In [21]:
```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import copy

SEED = 0

### CONSTANTS ###
BATCH_SIZE = 256
EPOCHS = 10
LR = 0.0001


class ResNet18(nn.Module):
    def __init__(self):
        super(ResNet18, self).__init__()
        self.relu = nn.ReLU()
        self.skip = nn.Identity()

        self.in_block = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            nn.ReLU()
        )

        self.block_1a = nn.Sequential(
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
        )

        self.block_1b = nn.Sequential(
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
        )

        self.block_2a = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(128),
        )
        self.conv_skip_2 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=1, stride=2, bias=False),
            nn.BatchNorm2d(128)
        )
```

```
        self.block_2b = nn.Sequential(
            nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.Conv2d(128, 128, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(128),
        )

        self.block_3a = nn.Sequential(
            nn.Conv2d(128, 256, kernel_size=3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(256),
        )
        self.conv_skip_3 = nn.Sequential(
            nn.Conv2d(128, 256, kernel_size=1, stride=2, bias=False),
            nn.BatchNorm2d(256),
        )

        self.block_3b = nn.Sequential(
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(256),
        )

        self.block_4a = nn.Sequential(
            nn.Conv2d(256, 512, kernel_size=3, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(),
            nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(512),
        )
        self.conv_skip_4 = nn.Sequential(
            nn.Conv2d(256, 512, kernel_size=1, stride=2, bias=False),
            nn.BatchNorm2d(512),
        )

        self.block_4b = nn.Sequential(
            nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(),
            nn.Conv2d(512, 512, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(512),
        )

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, 10)


    def forward(self, x):
        x = self.in_block(x)

        x = self.block_1a(x) + self.skip(x)
        x = self.relu(x)

        x = self.block_1b(x) + self.skip(x)
        x = self.relu(x)
```

```
        x = self.block_2a(x) + self.conv_skip_2(x)
        x = self.relu(x)
        x = self.block_2b(x) + self.skip(x)
        x = self.relu(x)

        x = self.block_3a(x) + self.conv_skip_3(x)
        x = self.relu(x)
        x = self.block_3b(x) + self.skip(x)
        x = self.relu(x)

        x = self.block_4a(x) + self.conv_skip_4(x)
        x = self.relu(x)
        x = self.block_4b(x) + self.skip(x)
        x = self.relu(x)

        x = self.avgpool(x)
        x = x.flatten(1)
        x = self.fc(x)

        return x


### MODEL ###
# Initialize the custom ResNet18 model
resnet18_model = ResNet18()
```

In [22]:

```
### DATA ###

# Transform to convert FMNIST data into normalized tensors
fmnist_data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])


# Init train and test datasets
train_data = datasets.FashionMNIST(
    root='/datasets/FMNIST/',
    train=True,
    download=True,
    transform=fmnist_data_transform
)

test_data = datasets.FashionMNIST(
    root="/datasets/FMNIST/",
    train=False,
    download=True,
    transform=fmnist_data_transform,
)

# Will need validation for early stopping. To do that, I split the training dataset
# 90% for training, 10% for validation
val_size = int(0.1 * len(train_data))
train_size = len(train_data) - val_size

train_set, val_set = torch.utils.data.random_split(
    train_data, [train_size, val_size], generator=torch.Generator().manual_seed(SEED)
)

# Define dataloaders
FMNIST_train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True)
```

```
FMNIST_val_loader = DataLoader(val_set, batch_size=BATCH_SIZE, shuffle=False)
FMNIST_test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
```

a) Training with L2 Regularization

In [23]:

```
### L2 Regularization TRAIN and TEST ###
LAMBDA = 0.0001


# Training with l2 regularization
def train_l2_regularize(
    epochs, learning_rate, model, train_loader, test_loader, lambda_val
):
    test_accs = []
    train_accs = []
    # Define the optimizer, use Adam with no weight decay (L2 regularization is added manually)
    optim = torch.optim.Adam(
        params=model.parameters(), lr=learning_rate, weight_decay=0
    )
    # The loss criterion is the cross entropy loss
    criterion = nn.CrossEntropyLoss()

    # Training loop
    for _ in range(epochs):
        total = 0
        correct = 0
        model.train()

        # Iterate over batches of data
        for features, labels in train_loader:
            # Calculate original cross entropy loss
            output_model = model(features)
            predictions = output_model.argmax(axis=1)
            loss = criterion(output_model, labels)
            total += output_model.size(0)
            correct += (predictions == labels).sum().item()
            # Add l2 regularization term defined as lambda / 2 * sum of squared parameters
            loss += lambda_val * sum((p**2).sum() for p in model.parameters()) / 2

            # Backpropagate
            optim.zero_grad()
            loss.backward()
            optim.step()

        train_acc = correct / total
        train_accs.append(train_acc)

        # Evaluate on the test set
        model.eval()

        total = 0
        correct = 0
        with torch.no_grad():
            for features, labels in test_loader:
                output_model_test = model(features)
                predictions = output_model_test.argmax(axis=1)
                total += output_model_test.size(0)
                correct += (predictions == labels).sum().item()

        test_acc = correct / total
```

```
        test_accs.append(test_acc)
    return train_accs, test_accs


l2_regularization_model = copy.deepcopy(resnet18_model)
train_accs_l2, test_accs_l2 = train_l2_regularize(
    EPOCHS, LR, l2_regularization_model, FMNIST_train_loader, FMNIST_test_loader, LAMBDA
)
```

In [24]:

```
# Plotting script
import matplotlib.pyplot as plt

def plot_accs(title, train_accs, test_accs):
    epochs = range(1, len(train_accs) + 1)

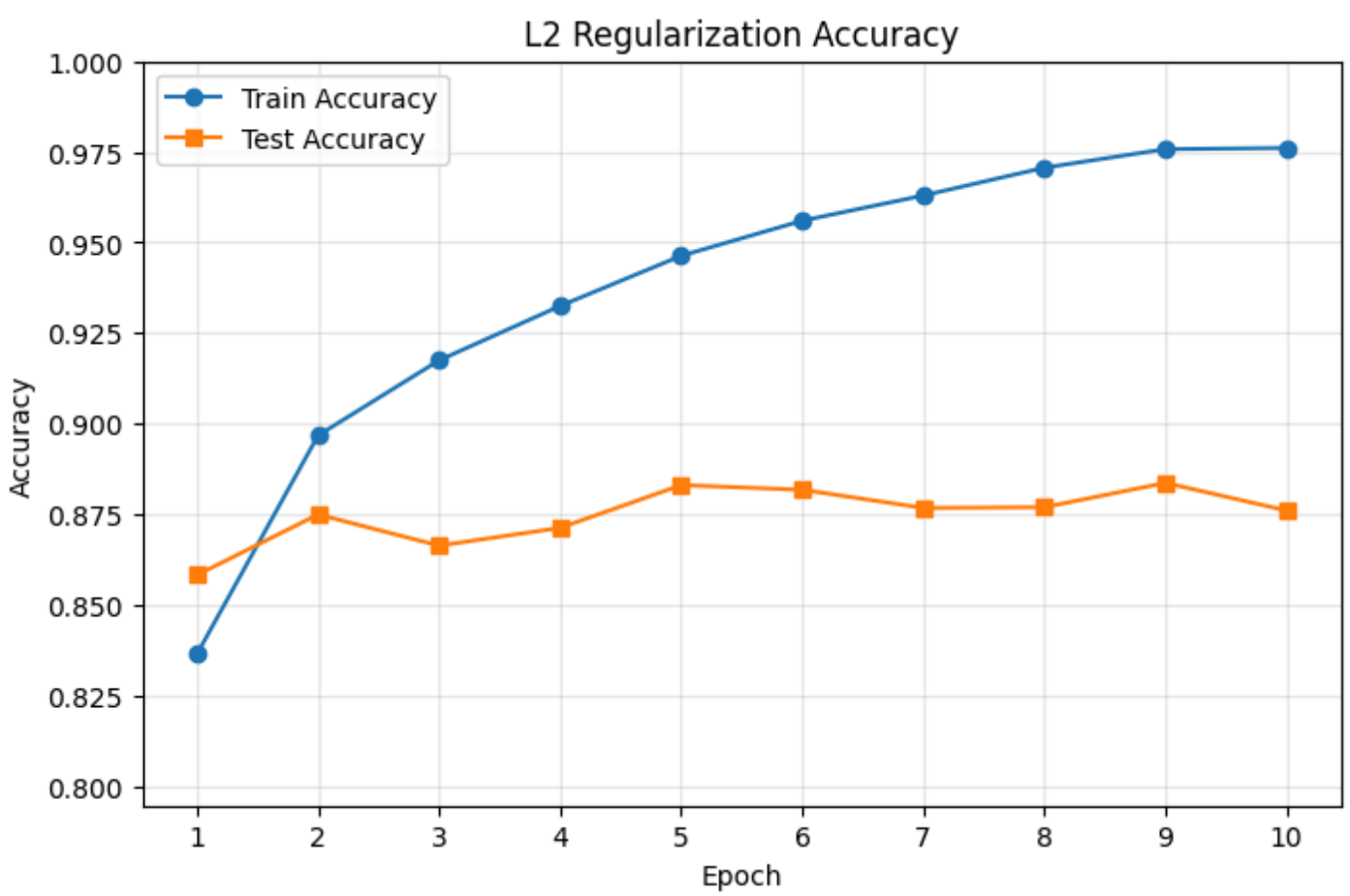
    plt.figure(figsize=(8, 5))

    plt.plot(epochs, train_accs, marker="o", label="Train Accuracy")
    plt.plot(epochs, test_accs, marker="s", label="Test Accuracy")

    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title(title)
    plt.xticks(epochs)
    plt.ylim(min(0.8, 0.95 * min(min(train_accs), min(test_accs))), 1)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

def print_stats(train_accs, test_accs):
    print(f'Final train accuracy: {train_accs[-1]}, average train accuracy: {sum(train_accs) / l
    print(f'Final test accuracy: {test_accs[-1]}, average test accuracy: {sum(test_accs) / len(t

plot_accs("L2 Regularization Accuracy", train_accs_l2, test_accs_l2)
print_stats(train_accs_l2, test_accs_l2)
```




```
Final train accuracy: 0.9760925925925926, average train accuracy: 0.9371222222222222
Final test accuracy: 0.8761, average test accuracy: 0.8749499999999999
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0.0001 = 10^{-4}$

Weight Decay of Adam: 0

b) Training with Adam with weight decay

In [25]:

```python
### Weight-Decay Adam TRAIN and TEST ###
weight_decay = LAMBDA


# Training with weight-decay Adam
def train_weight_decay(
    epochs, learning_rate, model, train_loader, test_loader, weight_decay_const
):
    test_accs = []
    train_accs = []
    # Define the optimizer, use Adam with weight decay
    optim = torch.optim.Adam(
        params=model.parameters(), lr=learning_rate, weight_decay=weight_decay_const
    )
    # The loss criterion is the cross entropy loss
    criterion = nn.CrossEntropyLoss()

    # Training loop
    for _ in range(epochs):
        total = 0
        correct = 0
```

```
        model.train()

        # Iterate over batches of data
        for features, labels in train_loader:
            # Calculate original cross entropy loss
            output_model = model(features)
            predictions = output_model.argmax(axis=1)
            loss = criterion(output_model, labels)
            total += output_model.size(0)
            correct += (predictions == labels).sum().item()

            # Backpropagate
            optim.zero_grad()
            loss.backward()
            optim.step()

        train_acc = correct / total
        train_accs.append(train_acc)

        # Evaluate on the test set
        model.eval()

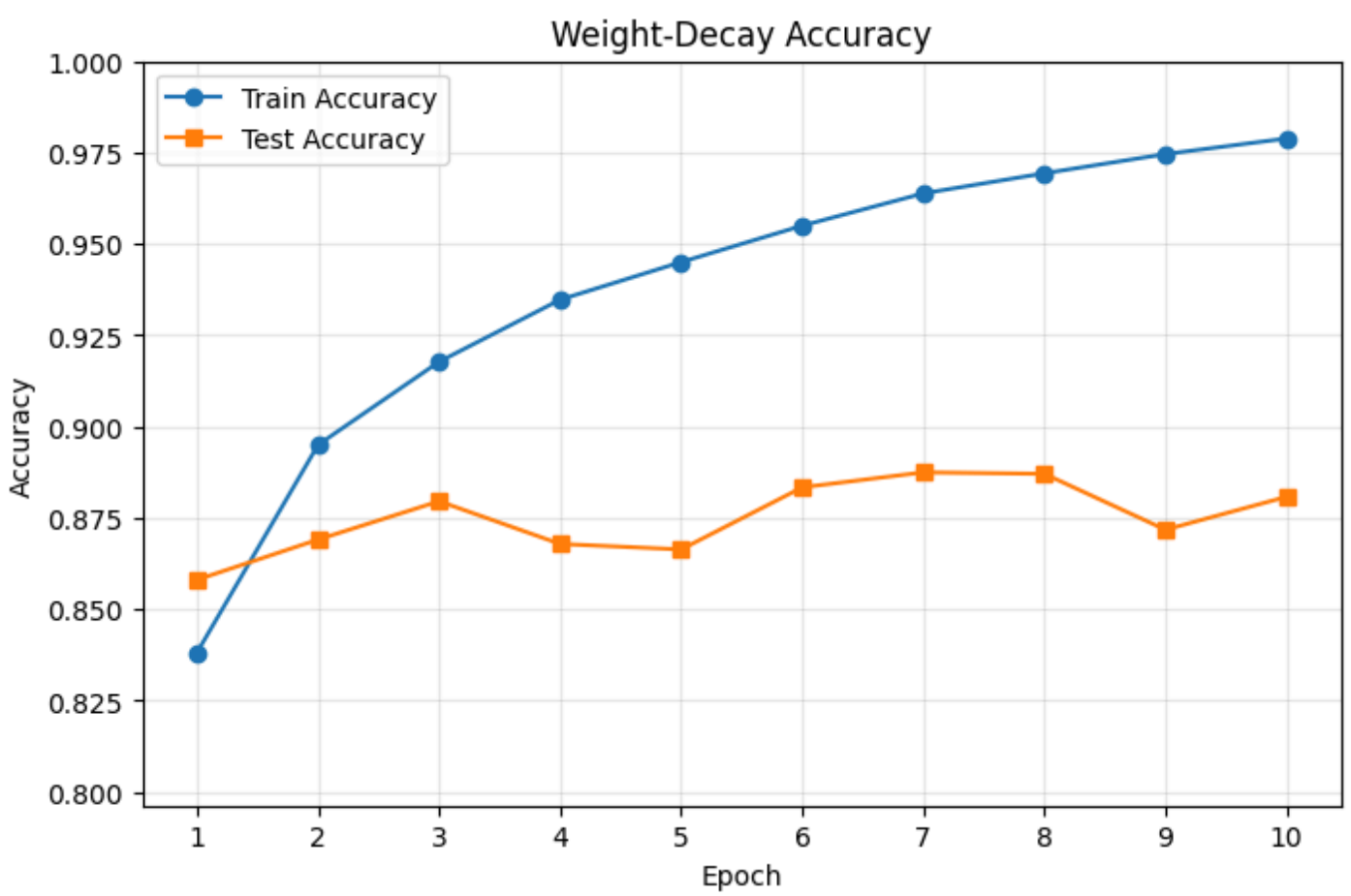
        total = 0
        correct = 0
        with torch.no_grad():
            for features, labels in test_loader:
                output_model_test = model(features)
                predictions = output_model_test.argmax(axis=1)
                total += output_model_test.size(0)
                correct += (predictions == labels).sum().item()

        test_acc = correct / total
        test_accs.append(test_acc)
    return train_accs, test_accs


weight_decay_model = copy.deepcopy(resnet18_model)
train_accs_wd, test_accs_wd = train_weight_decay(
    EPOCHS,
    LR,
    weight_decay_model,
    FMNIST_train_loader,
    FMNIST_test_loader,
    weight_decay,
)
```

In [26]:

```
plot_accs("Weight-Decay Accuracy", train_accs_wd, test_accs_wd)
print_stats(train_accs_wd, test_accs_wd)
```




```
Final train accuracy: 0.9788703703703704, average train accuracy: 0.9372518518518518
Final test accuracy: 0.8808, average test accuracy: 0.87517
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0$

Weight Decay of Adam: $0.0001 = 10^{-4}$

c) AdamW with weight decay

In [27]:
```python
### Weight-Decay AdamW TRAIN and TEST ###
weight_decay_AdamW = LAMBDA


# Training with weight-decay AdamW
def train_weight_decay_AdamW(
    epochs, learning_rate, model, train_loader, test_loader, weight_decay_const
):
    test_accs = []
    train_accs = []
    # Define the optimizer, use AdamW with weight decay
    optim = torch.optim.AdamW(
        params=model.parameters(), lr=learning_rate, weight_decay=weight_decay_const
    )
    # The loss criterion is the cross entropy loss
    criterion = nn.CrossEntropyLoss()

    # Training loop
    for _ in range(epochs):
        total = 0
        correct = 0
```

```
        model.train()

        # Iterate over batches of data
        for features, labels in train_loader:
            # Calculate original cross entropy loss
            output_model = model(features)
            predictions = output_model.argmax(axis=1)
            loss = criterion(output_model, labels)
            total += output_model.size(0)
            correct += (predictions == labels).sum().item()

            # Backpropagate
            optim.zero_grad()
            loss.backward()
            optim.step()

        train_acc = correct / total
        train_accs.append(train_acc)

        # Evaluate on the test set
        model.eval()

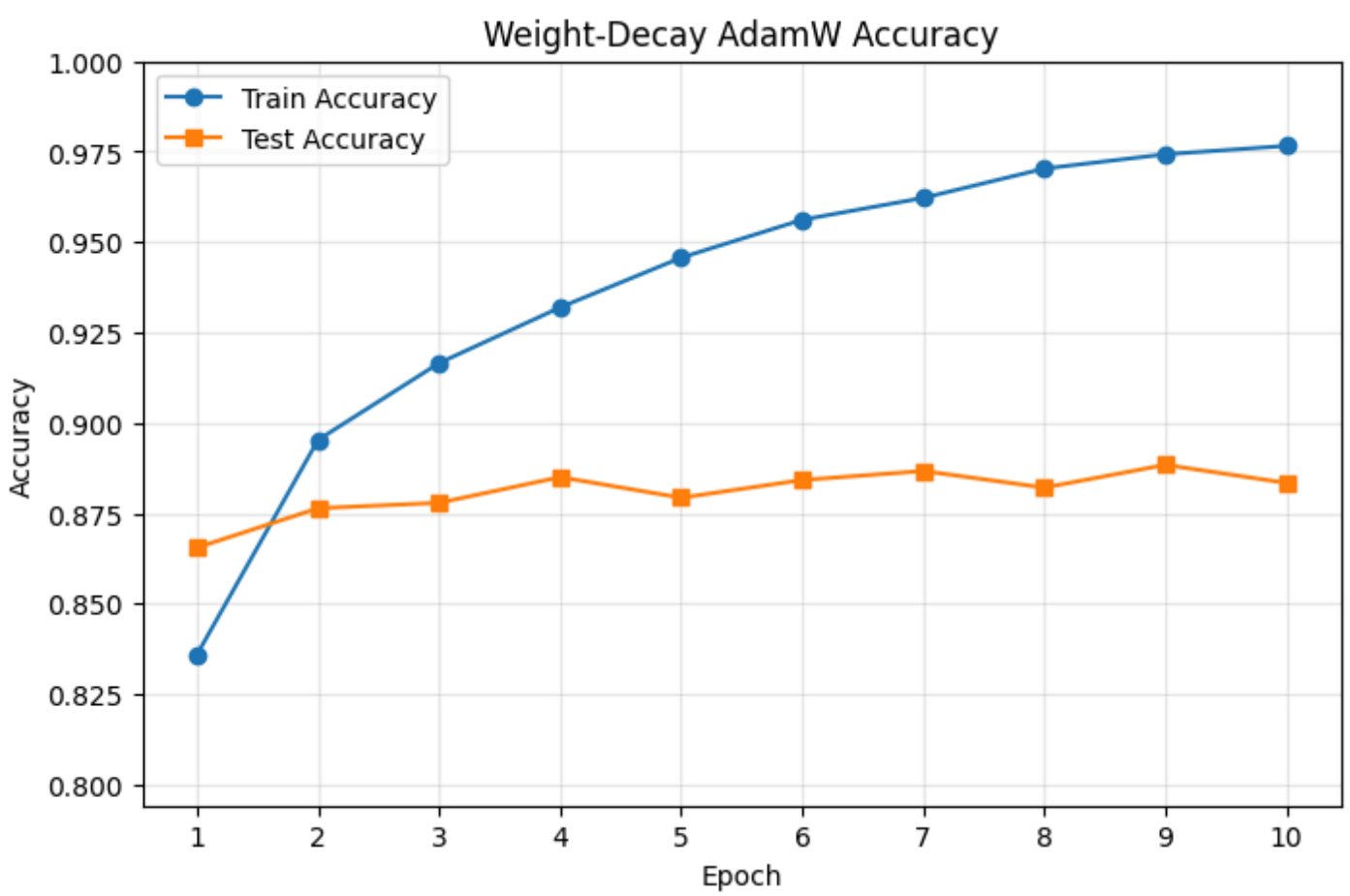
        total = 0
        correct = 0
        with torch.no_grad():
            for features, labels in test_loader:
                output_model_test = model(features)
                predictions = output_model_test.argmax(axis=1)
                total += output_model_test.size(0)
                correct += (predictions == labels).sum().item()

        test_acc = correct / total
        test_accs.append(test_acc)
    return train_accs, test_accs


weight_decay_AdamW_model = copy.deepcopy(resnet18_model)
train_accs_wd_AdamW, test_accs_wd_AdamW = train_weight_decay_AdamW(
    EPOCHS,
    LR,
    weight_decay_AdamW_model,
    FMNIST_train_loader,
    FMNIST_test_loader,
    weight_decay_AdamW,
)
```

In [28]:

```
plot_accs("Weight-Decay AdamW Accuracy", train_accs_wd_AdamW, test_accs_wd_AdamW)
print_stats(train_accs_wd_AdamW, test_accs_wd_AdamW)
```




```
Final train accuracy: 0.9766111111111111, average train accuracy: 0.9365111111111111
Final test accuracy: 0.8834, average test accuracy: 0.88089
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0$

Weight Decay of AdamW: $0.0001 = 10^{-4}$

d) All methods with early stopping

In [29]:

```python
### L2 Regularization TRAIN and TEST  with early stopping ###
LAMBDA = 0.0001


# Training with l2 regularization with early stopping
def train_l2_regularize_early_stop(
    epochs,
    learning_rate,
    model,
    train_loader,
    valid_loader,
    test_loader,
    lambda_val,
    patience=2,
):
    test_accs = []
    train_accs = []
    valid_loss = []

    best_val_loss = float("inf")
    epochs_without_better_valid_loss = 0
```

```
# Define the optimizer, use Adam with no weight decay (L2 regularization is added manually)
optim = torch.optim.Adam(
    params=model.parameters(), lr=learning_rate, weight_decay=0
)
# The loss criterion is the cross entropy loss
criterion = nn.CrossEntropyLoss()

# Training loop
for _ in range(epochs):
    total = 0
    correct = 0
    model.train()

    # Iterate over batches of data
    for features, labels in train_loader:
        # Calculate original cross entropy loss
        output_model = model(features)
        predictions = output_model.argmax(axis=1)
        loss = criterion(output_model, labels)
        total += output_model.size(0)
        correct += (predictions == labels).sum().item()
        # Add l2 regularization term defined as lambda / 2 * sum of squared parameters
        loss += lambda_val * sum((p**2).sum() for p in model.parameters()) / 2

        # Backpropagate
        optim.zero_grad()
        loss.backward()
        optim.step()

    train_acc = correct / total
    train_accs.append(train_acc)

    # Evaluate on the test set
    model.eval()

    total = 0
    correct = 0
    with torch.no_grad():
        for features, labels in test_loader:
            output_model_test = model(features)
            predictions = output_model_test.argmax(axis=1)
            total += output_model_test.size(0)
            correct += (predictions == labels).sum().item()

    test_acc = correct / total
    test_accs.append(test_acc)

    # Validation for early stopping
    total_val_loss = 0
    total_val_samples = 0

    # Calculate the average validation loss
    with torch.no_grad():
        for features, labels in valid_loader:
            output_model_val = model(features)
            val_loss = criterion(output_model_val, labels)
            total_val_loss += val_loss.item() * output_model_val.size(0)
            total_val_samples += output_model_val.size(0)

    avg_val_loss = total_val_loss / total_val_samples
    valid_loss.append(avg_val_loss)
```

```
        # Check if the current validation loss is the best one
        if avg_val_loss < best_val_loss:
            # Update the best validation loss
            best_val_loss = avg_val_loss
            epochs_without_better_valid_loss = 0
        else:
            # No improvement in validation loss
            epochs_without_better_valid_loss += 1
            # If the number of epochs without improvement exceeds the patience, stop early
            if epochs_without_better_valid_loss >= patience:
                print(f"Early stopping at epoch {_ + 1}")
                break

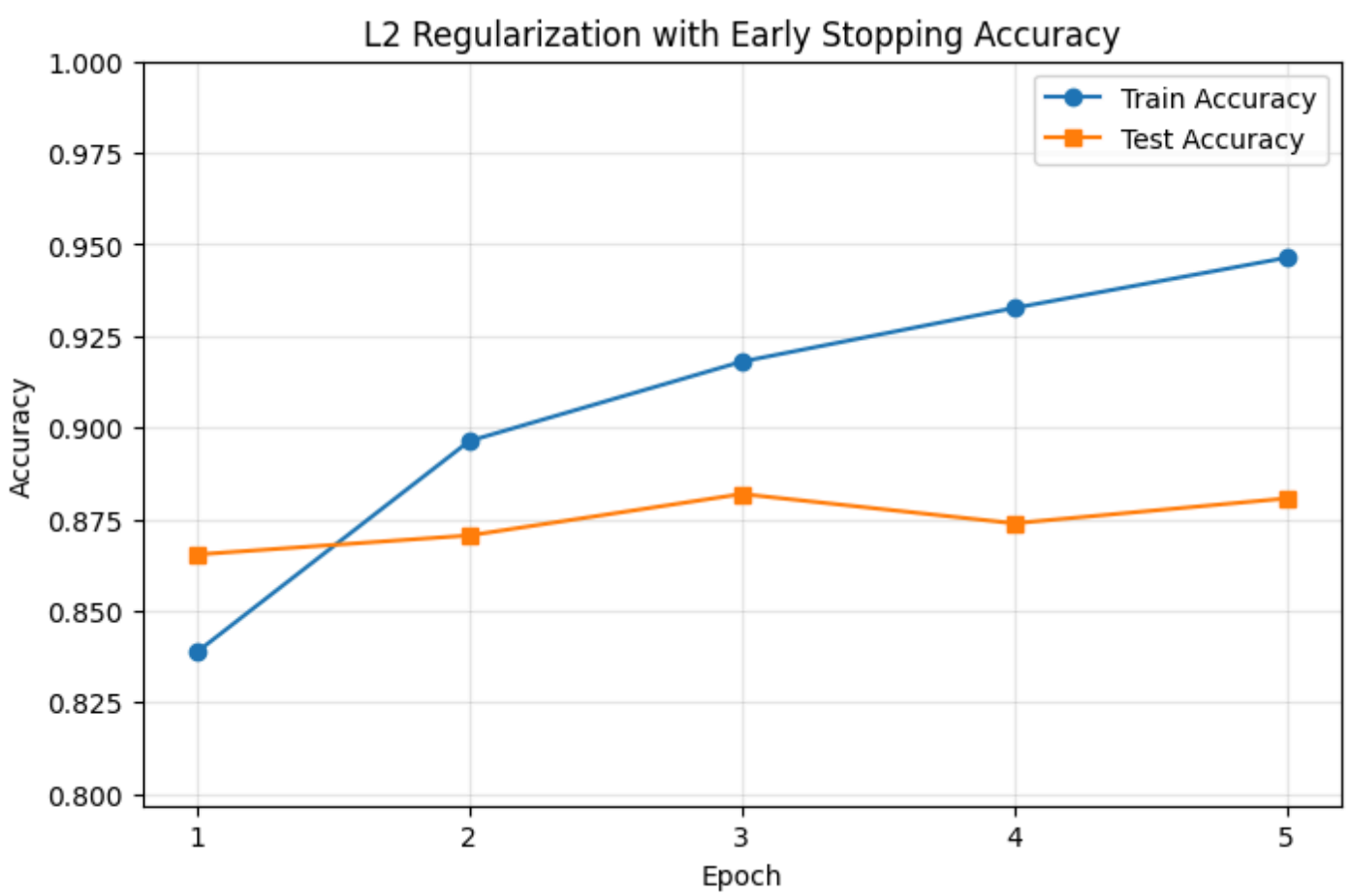
    return train_accs, test_accs, valid_loss


l2_regularization_model = copy.deepcopy(resnet18_model)
train_accs_l2_early_stop, test_accs_l2_early_stop, valid_loss_l2_early_stop = (
    train_l2_regularize_early_stop(
        EPOCHS,
        LR,
        l2_regularization_model,
        FMNIST_train_loader,
        FMNIST_val_loader,
        FMNIST_test_loader,
        LAMBDA,
    )
)
```

```
Early stopping at epoch 5
```

In [30]:

```
plot_accs(
    "L2 Regularization with Early Stopping Accuracy",
    train_accs_l2_early_stop,
    test_accs_l2_early_stop,
)
print_stats(train_accs_l2_early_stop, test_accs_l2_early_stop)
print(
    f"Final 2 validation losses: {valid_loss_l2_early_stop[-2]}, {valid_loss_l2_early_stop[-1]}"
)
```




```
Final train accuracy: 0.9463888888888888, average train accuracy: 0.9064518518518518
Final test accuracy: 0.8807, average test accuracy: 0.8745
Final 2 validation losses: 0.34542136764526365, 0.34091627748807274
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0.0001 = 10^{-4}$

Weight Decay of Adam: 0

Early stop triggered at epoch 5

In [31]:

```
### Weight-Decay Adam TRAIN and TEST with early stopping ###
weight_decay = LAMBDA

# Training with Weight-Decay Adam and early stopping
def train_weight_decay_early_stop(
    epochs,
    learning_rate,
    model,
    train_loader,
    valid_loader,
    test_loader,
    weight_decay_const,
    patience=2,
):
    test_accs = []
    train_accs = []
    valid_loss = []

    best_val_loss = float("inf")
    epochs_without_better_valid_loss = 0
```

```
# Define the optimizer, use Adam
optim = torch.optim.Adam(
    params=model.parameters(), lr=learning_rate, weight_decay=weight_decay_const
)
# The loss criterion is the cross entropy loss
criterion = nn.CrossEntropyLoss()

# Training loop
for _ in range(epochs):
    total = 0
    correct = 0
    model.train()

    # Iterate over batches of data
    for features, labels in train_loader:
        # Calculate original cross entropy loss
        output_model = model(features)
        predictions = output_model.argmax(axis=1)
        loss = criterion(output_model, labels)
        total += output_model.size(0)
        correct += (predictions == labels).sum().item()

        # Backpropagate
        optim.zero_grad()
        loss.backward()
        optim.step()

    train_acc = correct / total
    train_accs.append(train_acc)

    # Evaluate on the test set
    model.eval()

    total = 0
    correct = 0
    with torch.no_grad():
        for features, labels in test_loader:
            output_model_test = model(features)
            predictions = output_model_test.argmax(axis=1)
            total += output_model_test.size(0)
            correct += (predictions == labels).sum().item()

    test_acc = correct / total
    test_accs.append(test_acc)

    # Validation for early stopping
    total_val_loss = 0
    total_val_samples = 0

    # Calculate the average validation loss
    with torch.no_grad():
        for features, labels in valid_loader:
            output_model_val = model(features)
            val_loss = criterion(output_model_val, labels)
            total_val_loss += val_loss.item() * output_model_val.size(0)
            total_val_samples += output_model_val.size(0)

    avg_val_loss = total_val_loss / total_val_samples
    valid_loss.append(avg_val_loss)

    # Check if the current validation loss is the best one
    if avg_val_loss < best_val_loss:
```

```
            # Update the best validation loss
            best_val_loss = avg_val_loss
            epochs_without_better_valid_loss = 0
        else:
            # No improvement in validation loss
            epochs_without_better_valid_loss += 1
            # If the number of epochs without improvement exceeds the patience, stop early
            if epochs_without_better_valid_loss >= patience:
                print(f"Early stopping at epoch {_ + 1}")
                break

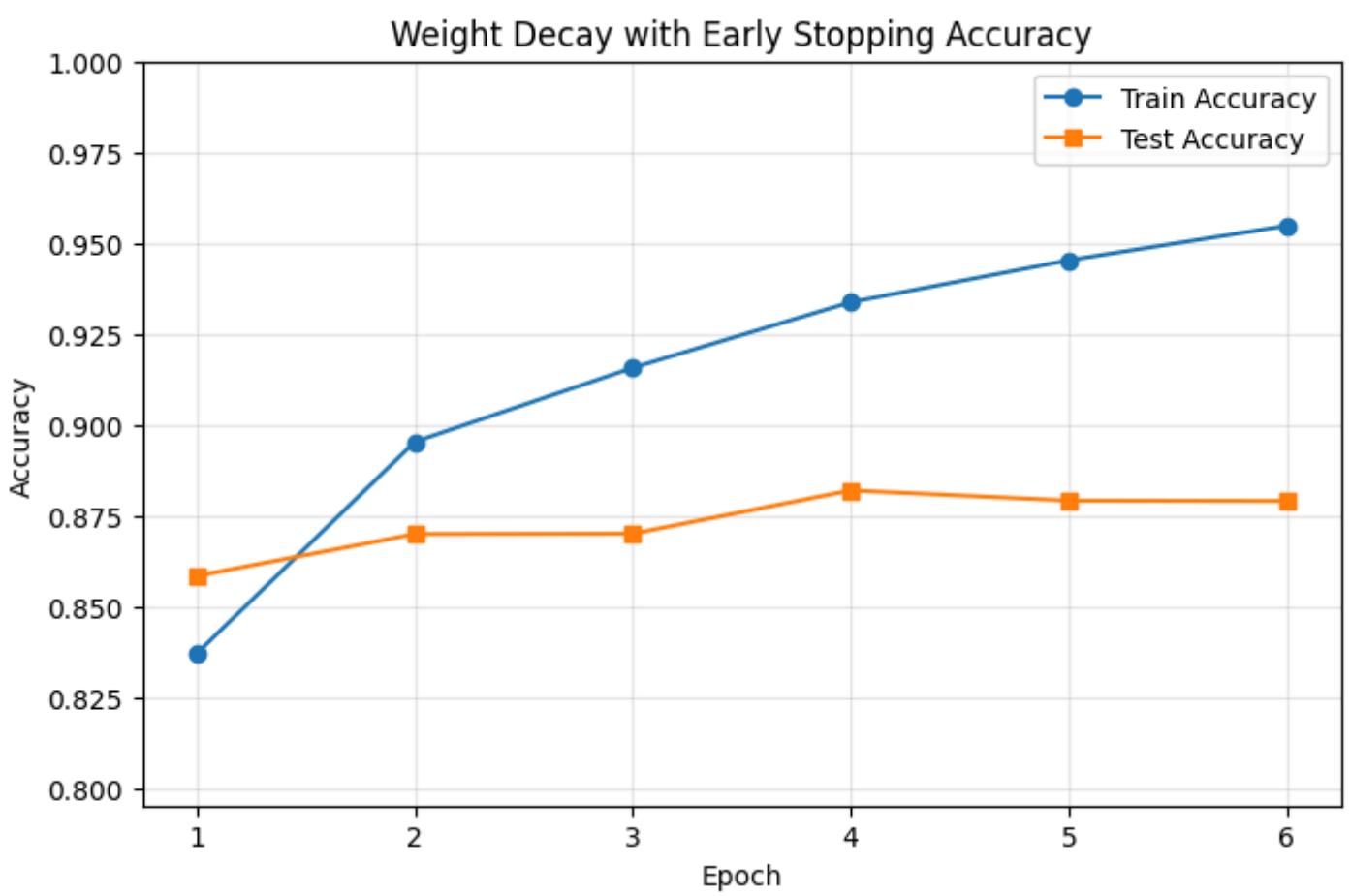
    return train_accs, test_accs, valid_loss


weight_decay_model = copy.deepcopy(resnet18_model)
(
    train_accs_weight_decay_early_stop,
    test_accs_weight_decay_early_stop,
    valid_loss_weight_decay_early_stop,
) = train_weight_decay_early_stop(
    EPOCHS,
    LR,
    weight_decay_model,
    FMNIST_train_loader,
    FMNIST_val_loader,
    FMNIST_test_loader,
    weight_decay,
)
```

```
Early stopping at epoch 6
```

In [32]:

```
plot_accs(
    "Weight Decay with Early Stopping Accuracy",
    train_accs_weight_decay_early_stop,
    test_accs_weight_decay_early_stop,
)
print_stats(train_accs_weight_decay_early_stop, test_accs_weight_decay_early_stop)
print(
    f"Final 2 validation losses: {valid_loss_weight_decay_early_stop[-2]}, {valid_loss_weight_de
)
```




```
Final train accuracy: 0.9547962962962963, average train accuracy: 0.9137623456790124
Final test accuracy: 0.8792, average test accuracy: 0.87325
Final 2 validation losses: 0.3498074787457784, 0.3766766368548075
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0$

Weight Decay of Adam: $0.0001 = 10^{-4}$

Early stop triggered at epoch 6

In [33]:

```
### Weight-Decay Adam TRAIN and TEST with early stopping ###
weight_decay = LAMBDA

# Training with Weight-Decay AdamW and early stopping
def train_weight_decay_AdamW_early_stop(
    epochs,
    learning_rate,
    model,
    train_loader,
    valid_loader,
    test_loader,
    weight_decay_const,
    patience=2,
):
    test_accs = []
    train_accs = []
    valid_loss = []

    best_val_loss = float("inf")
    epochs_without_better_valid_loss = 0
```

```
# Define the optimizer, use AdamW
optim = torch.optim.AdamW(
    params=model.parameters(), lr=learning_rate, weight_decay=weight_decay_const
)
# The loss criterion is the cross entropy loss
criterion = nn.CrossEntropyLoss()

# Training loop
for _ in range(epochs):
    total = 0
    correct = 0
    model.train()

    # Iterate over batches of data
    for features, labels in train_loader:
        # Calculate original cross entropy loss
        output_model = model(features)
        predictions = output_model.argmax(axis=1)
        loss = criterion(output_model, labels)
        total += output_model.size(0)
        correct += (predictions == labels).sum().item()

        # Backpropagate
        optim.zero_grad()
        loss.backward()
        optim.step()

    train_acc = correct / total
    train_accs.append(train_acc)

    # Evaluate on the test set
    model.eval()

    total = 0
    correct = 0
    with torch.no_grad():
        for features, labels in test_loader:
            output_model_test = model(features)
            predictions = output_model_test.argmax(axis=1)
            total += output_model_test.size(0)
            correct += (predictions == labels).sum().item()

    test_acc = correct / total
    test_accs.append(test_acc)

    # Validation for early stopping
    total_val_loss = 0
    total_val_samples = 0

    # Calculate the average validation loss
    with torch.no_grad():
        for features, labels in valid_loader:
            output_model_val = model(features)
            val_loss = criterion(output_model_val, labels)
            total_val_loss += val_loss.item() * output_model_val.size(0)
            total_val_samples += output_model_val.size(0)

    avg_val_loss = total_val_loss / total_val_samples
    valid_loss.append(avg_val_loss)

    # Check if the current validation loss is the best one
    if avg_val_loss < best_val_loss:
```

```
            # Update the best validation loss
            best_val_loss = avg_val_loss
            epochs_without_better_valid_loss = 0
        else:
            # No improvement in validation loss
            epochs_without_better_valid_loss += 1
            # If the number of epochs without improvement exceeds the patience, stop early
            if epochs_without_better_valid_loss >= patience:
                print(f"Early stopping at epoch {_ + 1}")
                break

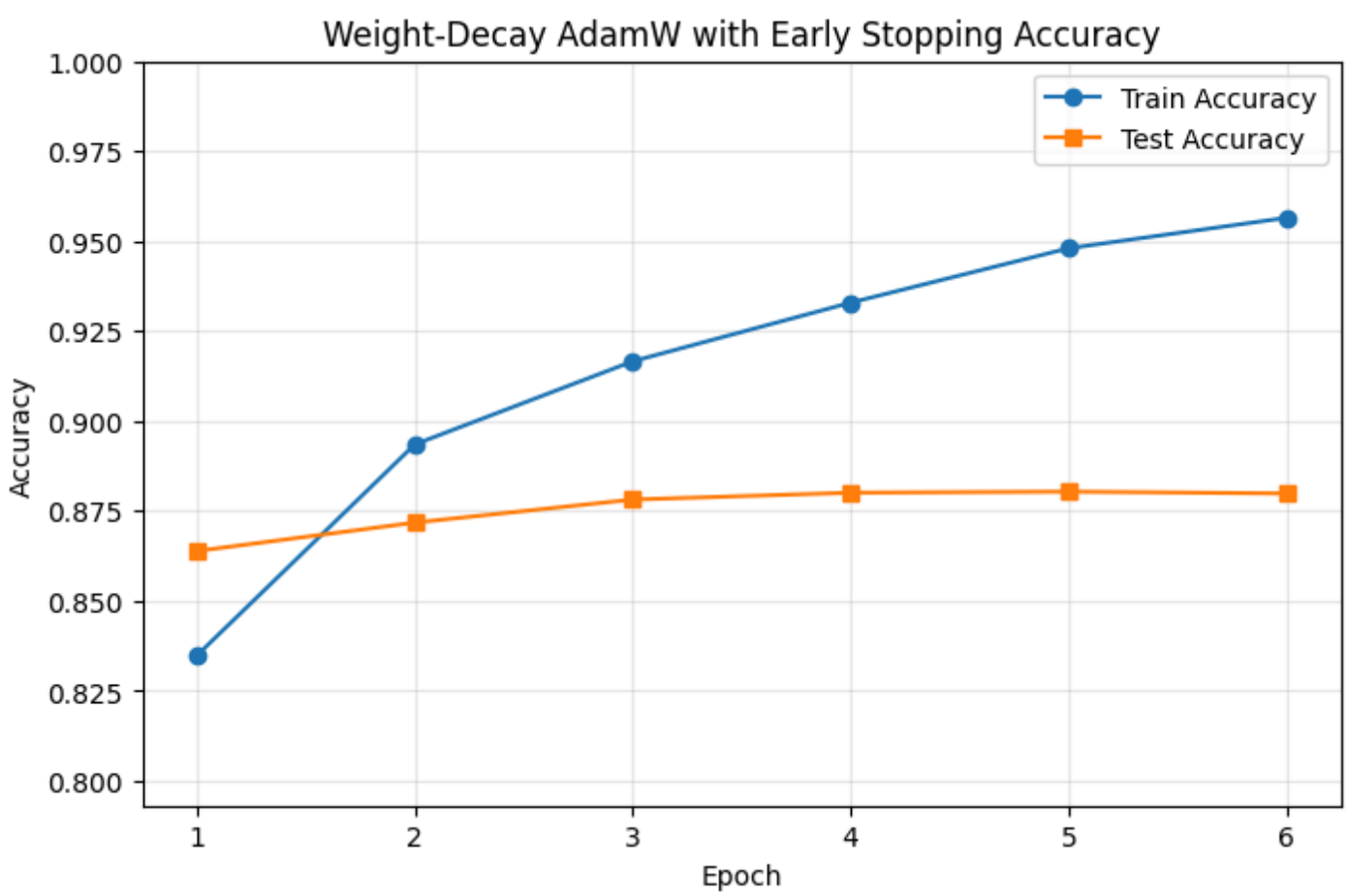
    return train_accs, test_accs, valid_loss


weight_decay_AdamW_model = copy.deepcopy(resnet18_model)
(
    train_accs_weight_decay_AdamW_early_stop,
    test_accs_weight_decay_AdamW_early_stop,
    valid_loss_weight_decay_AdamW_early_stop,
) = train_weight_decay_AdamW_early_stop(
    EPOCHS,
    LR,
    weight_decay_AdamW_model,
    FMNIST_train_loader,
    FMNIST_val_loader,
    FMNIST_test_loader,
    weight_decay,
)
```

```
Early stopping at epoch 6
```

In [34]:

```
plot_accs(
    "Weight-Decay AdamW with Early Stopping Accuracy",
    train_accs_weight_decay_AdamW_early_stop,
    test_accs_weight_decay_AdamW_early_stop,
)
print_stats(train_accs_weight_decay_AdamW_early_stop, test_accs_weight_decay_AdamW_early_stop)
print(
    f"Final 2 validation losses: {valid_loss_weight_decay_AdamW_early_stop[-2]}, {valid_loss_wei
)
```




```
Final train accuracy: 0.9565, average train accuracy: 0.9137530864197531
Final test accuracy: 0.8799, average test accuracy: 0.8757166666666666
Final 2 validation losses: 0.3366601941585541, 0.37001046641667684
```

Learning Rate $\eta = 0.0001 = 10^{-4}$

Regularization Hyperparameter $\lambda = 0$

Weight Decay of AdamW: $0.0001 = 10^{-4}$

Early stop triggered at epoch 6

Change in training and testing performance. I used early stopping with a patience of 2 (i.e. the training stops if the validation loss does not improve for 2 consecutive epochs), with 10% of the training data used for validation. L2 regularization stopped after 5 epochs, Adam with weight decay stopped after 6 epochs, and AdamW with weight decay also stopped after 6 epochs. The training and testing accuracies showed different behaviors for each method.

- L2 regularization: Training accuracy decreased from 97.61% to 94.64%, while test accuracy increased from 87.61% to 88.07%.
- Adam with weight decay: Training accuracy decreased from 97.89% to 95.48%, while test accuracy slightly decreased from 88.08% to 87.92%.
- AdamW with weight decay: Training accuracy decreased from 97.66% to 95.65%, while test accuracy slightly decreased from 88.34% to 87.99%.

In all cases, a notable decrease in computational cost (40%-50%) resulted in lower training accuracy while maintaining comparable testing performance.